DATA STRUCTURES AND PROGRAMMING PROJECT

Topic: Student Attendance Management

Group: I2-AMS-1
Team: 06

Group Members:	ID	Contribution
• CHENG Sokvisal	e20241146	100%
• HENG Limhong	e20241185	100%
• BOEUN Piseth	e20241245	100%

Lecturers: Mr. BOU CHANNA (Course)
Ms. CHOM SREYLAM (TP)

CONTENTS

1. | **Project Overview**
2. | **Objectives**
3. | **System Design**
4. | **Features & Functions**
5. | **Code Explanation**
6. | **Demonstration**
7. | **Conclusion**

PROJECT OVERVIEW

PURPOSE

- Automate attendance recording
- Reduce manual work and errors.
- Organize attendance records

TECHNOLOGY

- C++ as the main programming language
- Single linked list
- Stack for the undo function
- CSV file for handling the data
- Linear search
- Bubble sort method

BENEFITS

- Save time
- Improve accuracy and avoid duplicate and missing records
- Easy to manage and search for student

OBJECTIVES

MANAGE ATTENDANCE:

To develop a system that allows lecturers to record, update, and manage student attendance in a structured way.

STORE DATA:

To store all student attendance records permanently using CSV files, so that data can be saved and loaded anytime.

ORGANIZE RECORDS:

To provide functions for searching, sorting, and displaying student information clearly and efficiently.

APPLY PROGRAMMING CONCEPTS:

To practice and apply C++ concepts such as linked lists, stacks, file handling, and sorting algorithms in a real project.

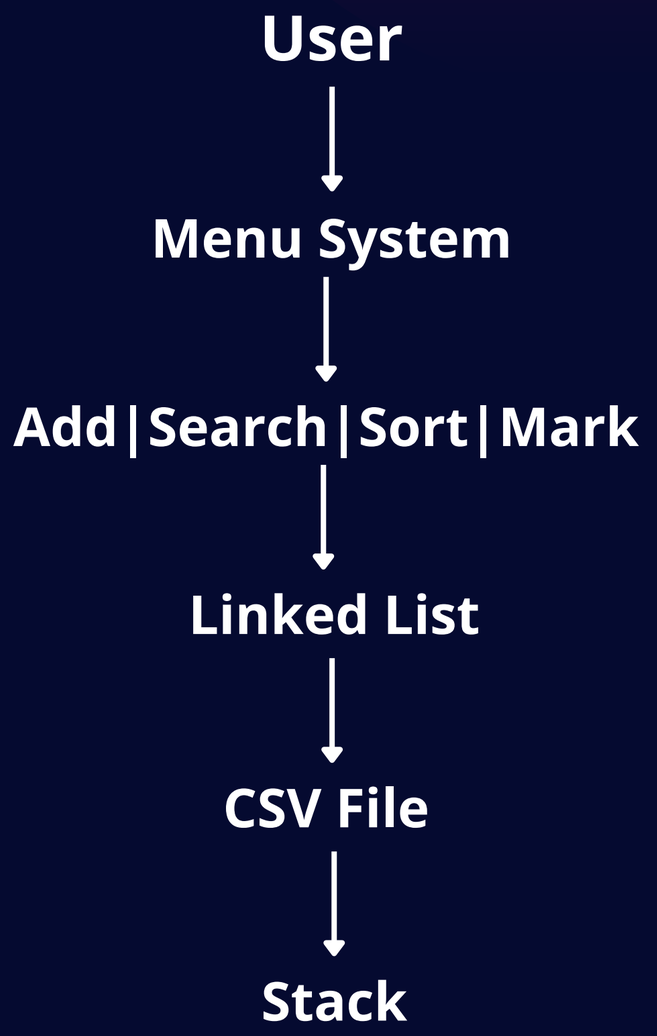
SYSTEM DESIGN +

User Interface
- Menu-driven console program

Data Storage
- Linked List (temporary storage)
- CSV File (permanent storage)

Core Functions
- Add, Search, Sort, Display

Advanced Features
- Stack-based Undo system



FEATURES & FUNCTIONS

Student Management

- Add student
- Display student
- Search student

Attendance Management

- Mark present/absent
- Update records
- Attendance rate

Data processing

- Search records
- Sort student of ID and name

File Management

- Load data from CSV file
- Save updated records to CSV file

Undo Feature

- Reverse last action using Stack

CODE EXPLANATION

1. Structures

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
using namespace std;

// ================= STRUCTURES =================
struct student{
    string ID;
    string name;
    int present;
    int absent;
};

struct node{
    student data;
    node* next;
};
```

```
struct Stack{
    undonode* top;
};
```

Explanation

Diagram:



Undo Stack:



CODE EXPLANATION

2. File Handling and CSV Storage

```
// ================= CSV FUNCTIONS =================
void loadCSV(List* ls){
    clearList(ls);

    ifstream fin("C:\\Users\\Admin\\Documents\\codeb

    if (!fin){
        cout << "Cannot open file!\n";
        return;
    }

    string line;
    getline(fin, line); // skip header
```

```
void saveCSV(List* ls){
    ofstream fout("C:\\Users\\Admin\\Docum

    if (!fout){
        cout << "Cannot open file!\n";
        return;
    }

    fout << "ID,Name,Present,Absent\n";

    node* temp = ls->head;
```

Explanation

CSV File

↓

loadCSV()

↓

Linked List

↓

saveCSV()

↓

CSV File

CODE EXPLANATION

3. Core Functions

+Add Student

```
// ================ STUDENT FUNCTIONS ================
void addStudent(List* ls){
    student s;

    cout << "Enter ID: ";
    cin >> s.ID;

    // check duplicate
    node* check = ls->head;
    while (check != NULL) {
        if (check->data.ID == s.ID){
            cout << "ID already exists!\n";
            return;
        }
        check = check->next;
    }

    cout << "Enter Name: ";
    cin >> s.name;

    s.present = 0;
    s.absent = 0;
```

+

```
    node* newnode = new node;
    newnode->data = s;
    newnode->next = NULL;

    if (ls->head == NULL)
        ls->head = newnode;
    else{
        node* temp = ls->head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newnode;
    }

    cout << "Student added!\n";
}
```

+Search Student

```
void searchByID(List* ls){
    if (ls->head == NULL){
        cout << "List is empty!\n";
        return;
    }

    string id;
    cout << "Enter Student ID: ";
    cin >> id;

    node* temp = ls->head;
    bool found = false;
```

```
void searchByName(List* ls){
    if (ls->head == NULL){
        cout << "List is empty!\n";
        return;
    }

    string name;
    cout << "Enter Student Name: ";
    cin >> name;

    node* temp = ls->head;
    bool found = false;

                << "\nSearch Results:\n";
```

```
void searchStudent(List* ls){
    int choice;

    cout << "\n===== SEARCH MENU =====\n";
    cout << "1. Search by ID\n";
    cout << "2. Search by Name\n";
    cout << "Enter choice: ";
    cin >> choice;

    switch (choice){
    case 1:
        searchByID(ls);
        break;

    case 2:
        searchByName(ls);
        break;

    default:
        cout << "Invalid choice!\n";
    }
}
```

CODE EXPLANATION

+Mark Attendance

```
// ================ ATTENDANCE ================
void markAttendance(List* ls, Stack* st){
    string id;
    cout << "Enter ID: ";
    cin >> id;

    node* temp = findStudent(ls, id);

    if (temp == NULL){
        cout << "Not found!\n";
        return;
    }

    pushUndo(st, temp->data.ID, temp->data.present, temp->data.absent);

    int choice;
    cout << "1. Present\n2. Absent\nChoice: ";
    cin >> choice;
```

Purpose

+Add Student:
- Adds a new student record.
- Checks for duplicate IDs.
- Stores the student in the linked list.

+Search Student:
- Search by ID or Name.
- Quickly locate student records.
- Displays attendance information.

+Mark Attendance:
- Marks students as Present or Absent.
- Updates attendance counts.
- Saves previous data for Undo.

CODE EXPLANATION

4. Algorithms & Advanced Features

+Linear Search

```
while (temp != NULL){
    if (temp->data.ID == id){
        cout << "\nStudent Found:\n";
        cout << "ID: " << temp->data.ID << endl;
        cout << "Name: " << temp->data.name << endl;
        cout << "Present: " << temp->data.present << endl;
        cout << "Absent: " << temp->data.absent << endl;

        found = true;
        break;
    }
    temp = temp->next;
}
```

```
while (temp != NULL){
    if (temp->data.name == name){
        cout << "ID: " << temp->data.ID << endl;
        cout << "Name: " << temp->data.name << endl;
        cout << "Present: " << temp->data.present << endl;
        cout << "Absent: " << temp->data.absent << endl;
        cout << "----------------------\n";

        found = true;
    }
    temp = temp->next;
}
```

+Sorting

```
void sortByID(List* ls){
    for (node* i = ls->head; i != NULL; i = i->next)
        for (node* j = i->next; j != NULL; j = j->next)
            if (i->data.ID > j->data.ID)
                swap(i->data, j->data);

    cout << "Sorted by ID!\n";
}
```

```
void sortByName(List* ls){
    for (node* i = ls->head; i != NULL; i = i->next)
        for (node* j = i->next; j != NULL; j = j->next)
            if (i->data.name > j->data.name)
                swap(i->data, j->data);

    cout << "Sorted by Name!\n";
}
```

student node::data

```
void sortByAttendance(List* ls){
    for (node* i = ls->head; i != NULL; i = i->next)
        for (node* j = i->next; j != NULL; j = j->next)
            if (i->data.present < j->data.present)
                swap(i->data, j->data);

    cout << "Sorted by Attendance!\n";
}
```

CODE EXPLANATION

+Undo Feature

```
    pushUndo(st, temp->data.ID, temp->data.present, temp->data.absent);

    int choice;
    cout << "1. Present\n2. Absent\nChoice: ";
    cin >> choice;

    if (choice == 1){
        temp->data.present++;
        cout << "Marked Present!\n";
    }
    else if (choice == 2){
        temp->data.absent++;
        cout << "Marked Absent!\n";
    }
    else{
        cout << "Invalid!\n";
    }
}
```

Purpose

+Linear Search:

- Traverses the linked list node by node.
- Searches for a student using ID or name.
- Returns the matching record when found.

+Sorting:

- Sorts student records.
- Supports sorting by: ID, Name and Attendance
- Uses comparison and swapping.

+Undo Features:

- Saves the previous attendance state.
- Uses a Stack data structure.
- Allows users to undo the most recent attendance update.

DEMONSTRATION +

CSV File

	A	B	C	D	E
1	ID	Name	Present	Absent	
2	1	Visal	26	4	
3	2	Piseth	24	6	
4	3	Hong	30	0	
5	4	Hour	30	0	
6	5	Heng	20	10	
7	6	Hak	19	11	
8	7	Heab	28	2	
9	8	Hean	20	10	
10	9	Han	15	15	
11	10	Xing	14	16	
12	11	Sok	0	0	
13					

Menu (CSV File loaded)

```
===== MENU =====
1. Load CSV
2. Add Student
3. Display
4. Search
5. Mark Attendance
6. Sort
7. Undo
8. Save CSV
9. Exit
Choice: 1
CSV loaded successfully!
```

DEMONSTRATION +

Display CSV File

```
===== MENU =====
1. Load CSV
2. Add Student
3. Display
4. Search
5. Mark Attendance
6. Sort
7. Undo
8. Save CSV
9. Exit
Choice: 3

ID      Name    Present Absent  Rate
----------------------------------------
01      Visal   26      4       86.6667%
02      Piseth  24      6       80%
03      Hong    30      0       100%
04      Hour    30      0       100%
05      Heng    20      10      66.6667%
06      Hak     19      11      63.3333%
07      Heab    28      2       93.3333%
08      Hean    20      10      66.6667%
09      Han     15      15      50%
10      Xing    14      16      46.6667%
11      Sok     0       0       0%
```

Add Student, Mark Attendance and Display the result

```
===== MENU =====
1. Load CSV
2. Add Student
3. Display
4. Search
5. Mark Attendance
6. Sort
7. Undo
8. Save CSV
9. Exit
Choice: 2
Enter ID: 12
Enter Name: Chen
Student added!
```

```
Choice: 5
Enter ID: 12
1. Present
2. Absent
Choice: 1
Marked Present!
```

```
ID      Name    Present Absent  Rate
----------------------------------------
01      Visal   26      4       86.6667%
02      Piseth  24      6       80%
03      Hong    30      0       100%
04      Hour    30      0       100%
05      Heng    20      10      66.6667%
06      Hak     19      11      63.3333%
07      Heab    28      2       93.3333%
08      Hean    20      10      66.6667%
09      Han     15      15      50%
10      Xing    14      16      46.6667%
11      Sok     0       0       0%
12      Chen    1       0       100%
```

DEMONSTRATION +

Search by ID and Name

```
Choice: 4

===== SEARCH MENU =====
1. Search by ID
2. Search by Name
Enter choice:
```

```
Enter choice: 1
Enter Student ID: 01

Student Found:
ID: 01
Name: Visal
Present: 26
Absent: 4
```

```
Enter choice: 2
Enter Student Name: Chen

Search Results:
ID: 12
Name: Chen
Present: 1
Absent: 0
```

Sort by Name and Attendance

ID	Name	Present	Absent	Rate
12	Chen	1	0	100%
06	Hak	19	11	63.3333%
09	Han	15	15	50%
07	Heab	28	2	93.3333%
08	Hean	20	10	66.6667%
05	Heng	20	10	66.6667%
03	Hong	30	0	100%
04	Hour	30	0	100%
02	Piseth	24	6	80%
11	Sok	0	0	0%
01	Visal	26	4	86.6667%
10	Xing	14	16	46.6667%

ID	Name	Present	Absent	Rate
03	Hong	30	0	100%
04	Hour	30	0	100%
07	Heab	28	2	93.3333%
01	Visal	26	4	86.6667%
02	Piseth	24	6	80%
08	Hean	20	10	66.6667%
05	Heng	20	10	66.6667%
06	Hak	19	11	63.3333%
09	Han	15	15	50%
10	Xing	14	16	46.6667%
12	Chen	1	0	100%
11	Sok	0	0	0%

DEMONSTRATION +

Undo

```
ID      Name    Present Absent  Rate
-------------------------------------
03      Hong    30      0       100%
04      Hour    30      0       100%
07      Heab    28      2       93.3333%
01      Visal   26      4       86.6667%
02      Piseth  24      6       80%
08      Hean    20      10      66.6667%
05      Heng    20      10      66.6667%
06      Hak     19      11      63.3333%
09      Han     15      15      50%
10      Xing    14      16      46.6667%
12      Chen    0       0       0%
11      Sok     0       0       0%
```

Save CSV File

```
Choice: 8
Saved successfully!
```

	A	B	C	D	E
1	ID	Name	Present	Absent	
2	3	Hong	30	0	
3	4	Hour	30	0	
4	7	Heab	28	2	
5	1	Visal	26	4	
6	2	Piseth	24	6	
7	8	Hean	20	10	
8	5	Heng	20	10	
9	6	Hak	19	11	
10	9	Han	15	15	
11	10	Xing	14	16	
12	12	Chen	0	0	
13	11	Sok	0	0	
14					

CONCLUSION

- Successfully developed a Student Attendance Management System using C++.
- The system can add, search, sort, and manage student attendance efficiently.
- Implemented important data structures such as Linked List and Stack.
- Used CSV file handling for permanent data storage.
- Improved understanding of real-world application of C++ programming.

REFERENCES +

- **Moodle (School Platform): Lecture notes, Coding tutorials and Assignment guidelines.**
- **W3school: To practice Coding.**
- **AI Assistance: ChatGPT, Grok and Gemini (Debugging and Code Explanation Support).**

WE APPRECIATE YOUR TIME!